MGPEB: Módulo de Gerenciamento de Pouso e Estabilização de Base

Missão Aurora Siger | Atividade Integradora, Fase 2 | FIAP

Grupo 3: Jean Melo

Outubro de 2026

1. Introdução

Após meses de viagem, a nave Aurora Siger entra na órbita de Marte e precisa enviar à superfície os módulos que formarão a base da colônia. Este relatório documenta o MGPEB, o sistema responsável por organizar a fila de pouso desses módulos, decidir quem pode descer e em que ordem, e registrar cada decisão para auditoria. O protótipo foi implementado em Python puro (sem bibliotecas externas) e está no arquivo mgpeb.py, entregue junto com este relatório. O trabalho articula os conteúdos da fase: portas lógicas, estruturas lineares, algoritmos de busca e ordenação, modelagem de funções, história da computação e princípios ESG.

2. Cenário de pouso e módulos

A base Aurora será montada a partir de seis módulos automatizados, desacoplados da nave mãe e enviados a três zonas de pouso (A, B e C). Cada módulo carrega os atributos que o MGPEB usa nas decisões: prioridade de pouso (1 é a máxima), nível de combustível, massa, criticidade da carga, horário estimado de chegada à órbita, integridade dos sensores e zona de destino.

ID	Módulo	Carga	Prior.	Comb. (%)	Massa (t)	Critic.	Chegada	Sensores	Zona
M01	Habitação	HABITAT	1	62,0	18,5	ALTA	T+12 min	OK	A
M02	Energia	ENERGIA	1	48,0	14,2	ALTA	T+8 min	OK	B
M03	Laboratório	CIÊNCIA	3	71,5	9,8	MÉDIA	T+25 min	OK	C
M04	Logística	LOGÍSTICA	4	23,0	21,0	BAIXA	T+15 min	OK	A
M05	Suporte Médico	MÉDICO	2	55,0	11,3	ALTA	T+20 min	FALHA	B
M06	Comunicações	COMUNICAÇÃO	2	30,0	7,6	MÉDIA	T+5 min	OK	C

Tabela 1. Módulos de pouso da base Aurora (destaque: valores que violam as regras).

O ambiente de pouso é simulado por quatro variáveis: vento (35 km/h, limite 60 km/h), tempestade de poeira (ausente), zonas livres (A e B livres, C ocupada por um módulo anterior) e sinal de emergência na base (desligado). O combustível mínimo para a manobra de descida é 30%.

Os módulos são organizados em uma **fila principal** de espera por autorização e em três **listas auxiliares**: pousados, em espera (pouso adiado) e em alerta (pouso abortado). Cada decisão é registrada em uma **pilha de eventos**, do evento mais recente para o mais antigo, que serve de trilha de auditoria. O Anexo A detalha o uso de cada estrutura.

3. Regras de decisão com portas lógicas

O MGPEB reduz a decisão de pouso a cinco entradas booleanas, obtidas da telemetria:

Sinal	Significado	Condição
C	combustível suficiente	combustível ≥ 30 %
A	atmosfera segura	vento ≤ 60 km/h AND NOT tempestade de poeira
Z	zona de pouso livre	a zona de destino do módulo não está ocupada
S	sensores íntegros	nenhuma falha de sensor reportada
E	emergência na base	sinal de emergência ativo

Tabela 2. Sinais booleanos de entrada do MGPEB.

As três saídas do sistema são funções booleanas dessas entradas:

- **AUTORIZAR** = C · A · Z · S · E’ (todas as condições satisfeitas e nenhuma emergência)
- **ADIAR** = AUTORIZAR’ · C (não pode pousar agora, mas tem combustível para aguardar em órbita)
- **ABORTAR** = C’ (sem combustível para aguardar: o módulo é desviado para a órbita de segurança)

As saídas são mutuamente exclusivas: para qualquer combinação de entradas, exatamente uma delas é verdadeira. A Figura 1 representa o circuito.

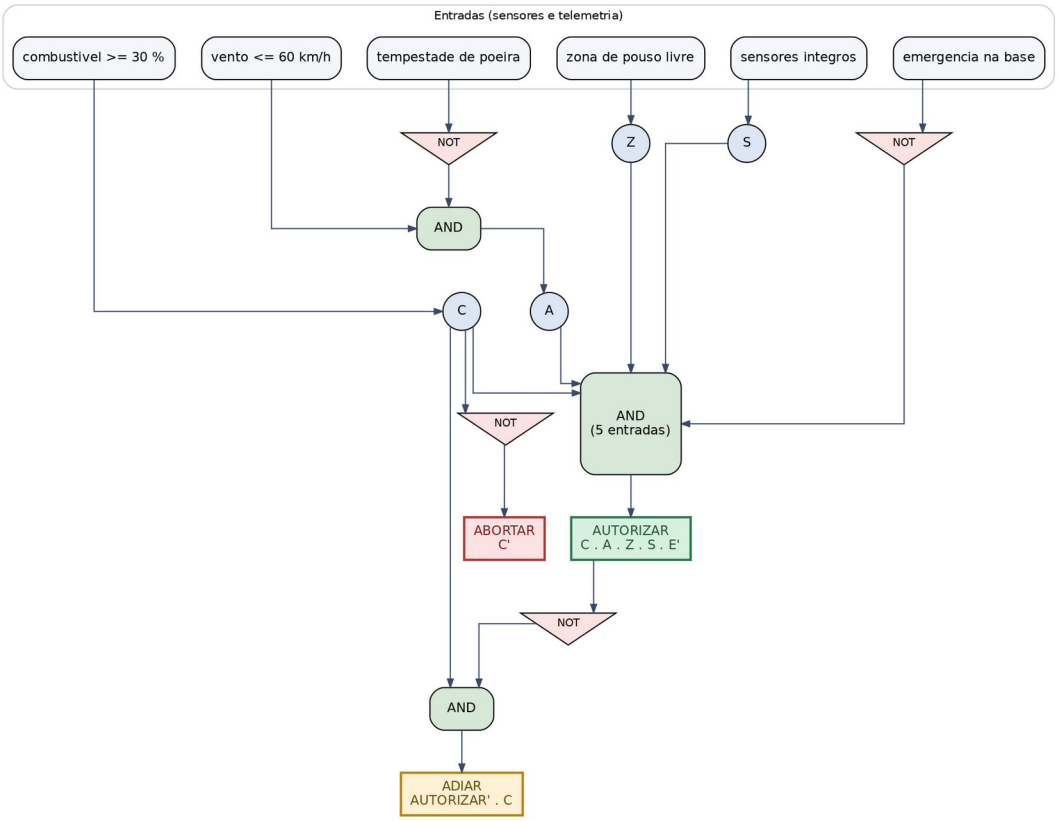


Figura 1. Circuito de decisão de pouso com portas AND e NOT.

A tabela verdade abaixo mostra os casos mais relevantes (A e E fixados em 1 e 0, que é o cenário simulado):

C	Z	S	AUTORIZAR	ADIAR	ABORTAR	Exemplo no cenário
1	1	1	1	0	0	M01, M02
1	0	1	0	1	0	M03, M06 (zona C ocupada)
1	1	0	0	1	0	M05 (falha de sensor)
1	0	0	0	1	0	M05 após a zona B ser ocupada
0	x	x	0	0	1	M04 (23 % de combustível)

Tabela 3. Tabela verdade dos casos do cenário (A = 1, E = 0).

No código, essas funções aparecem literalmente: autorizar = C and A and Z and S and (not E), seguido de um bloco if autorizar / elif C / else que devolve AUTORIZADO, ADIADO ou ABORTADO. A simplificação de ABORTAR para C’ decorre do fato de que, sem combustível, nenhuma outra condição importa: o módulo não consegue nem pousar nem esperar.

4. Protótipo em Python

O arquivo mgpeb.py tem cerca de 300 linhas, sem nenhum import, e está organizado em sete blocos: estruturas lineares (classes Fila e Pilha), cenário (lista MODULOS e dicionário AMBIENTE), buscas, ordenações, regras de decisão, modelo matemático e simulação principal. Os algoritmos foram escritos à mão, sem usar sorted() ou min(), para que o código reflita o conteúdo da disciplina.

Buscas. busca_sequencial percorre a lista inteira e devolve todos os módulos com um atributo igual ao valor pedido (por exemplo, criticidade ALTA). busca_extremo encontra o menor ou o maior valor de um atributo em uma passagem (menor combustível, maior prioridade). busca_binaria localiza um módulo pelo id em uma lista já ordenada, dividindo o intervalo pela metade a cada passo. As duas primeiras têm custo O(n); a binária, O(log n), mas exige a ordenação prévia.

Ordenações. ordenacao_selecao escolhe o menor elemento restante a cada passo e o move para a posição correta. ordenacao_insercao insere cada elemento na posição certa da parte já ordenada. Ambas são $O(n^2)$, adequadas para uma fila de poucos módulos. A inserção é estável (mantém a ordem relativa de empates), o que o MGPEB explora para montar a fila final: ordena por combustível e, em seguida, por prioridade. O resultado é uma fila ordenada por prioridade em que, dentro da mesma prioridade, o módulo com menos combustível desce primeiro.

Simulação. A função main() cadastra os módulos, executa as buscas e ordenações, monta a fila final, avalia cada módulo com avaliar_pouso e o move para a lista auxiliar correspondente. Quando um pouso é autorizado, a zona passa a ocupada, o que afeta as decisões seguintes. A Figura 2 mostra a execução completa.

Terminal | FIAP-Fase02xAuroraSiger

```
$ python3 src/mgpeb.py

======================================================================
 MGPEB | MODULOS CADASTRADOS (lista)
======================================================================
  M01 Habitacao       prio=1 comb= 62.0% massa=18.5t crit=ALTA  chegada=T+12min
  M02 Energia         prio=1 comb= 48.0% massa=14.2t crit=ALTA  chegada=T+ 8min
  M03 Laboratorio     prio=3 comb= 71.5% massa= 9.8t crit=MEDIA chegada=T+25min
  M04 Logistica       prio=4 comb= 23.0% massa=21.0t crit=BAIXA chegada=T+15min
  M05 Suporte Medico  prio=2 comb= 55.0% massa=11.3t crit=ALTA  chegada=T+20min
  M06 Comunicacoes    prio=2 comb= 30.0% massa= 7.6t crit=MEDIA chegada=T+ 5min

======================================================================
 BUSCAS
======================================================================
  Menor combustivel (busca sequencial): M04 Logistica (23.0%)
  Maior prioridade (busca sequencial):  M01 Habitacao (prioridade 1)
  Carga de criticidade ALTA:            M01, M02, M05
  Busca binaria por id M05:             Suporte Medico

======================================================================
 ORDENACAO DA FILA DE POUSO
======================================================================
  Por ordem de chegada (insercao):  M06 -> M02 -> M01 -> M04 -> M05 -> M03
  Por prioridade (selecao):         M01 -> M02 -> M05 -> M06 -> M03 -> M04
  Por combustivel, menor primeiro:  M04 -> M06 -> M02 -> M05 -> M01 -> M03
  Fila final (prioridade, depois combustivel): M02 -> M01 -> M06 -> M05 -> M03 -> M04

======================================================================
 AUTORIZACAO DE POUSO (fila + regras booleanas)
======================================================================
  Ambiente: vento=35.0 km/h, tempestade=False, zonas livres=['A', 'B']
  [OK]  M02 Energia         AUTORIZADO  (zona B)
  [OK]  M01 Habitacao       AUTORIZADO  (zona A)
  [..]  M06 Comunicacoes    ADIADO      (zona C ocupada)
  [..]  M05 Suporte Medico  ADIADO      (zona B ocupada; falha nos sensores)
  [..]  M03 Laboratorio     ADIADO      (zona C ocupada)
  [!!]  M04 Logistica       ABORTADO    (combustivel abaixo do minimo; zona A ocupada)

  Pousados:   ['M02', 'M01']
  Em espera:  ['M06', 'M05', 'M03']
  Em alerta:  ['M04']

======================================================================
 REGISTRO DE EVENTOS (pilha, do mais recente ao mais antigo)
======================================================================
  Ultimo evento (topo): ('M04', 'ABORTADO')
  ('M04', 'ABORTADO')
  ('M03', 'ADIADO')
  ('M05', 'ADIADO')
  ('M06', 'ADIADO')
  ('M01', 'AUTORIZADO')
  ('M02', 'AUTORIZADO')

======================================================================
 MODELO: ALTITUDE DURANTE A DESCIDA  h(t) = h0 - v0*t - (g/2)*t^2
======================================================================
  h0=2000 m, v0=60 m/s, g=3.71 m/s^2, acionamento em h=500 m
  t= 0 s -> h=  2000.0 m
```

Figura 2. Execução do protótipo (python3 mgpeb.py).

Resultado: M02 (Energia) e M01 (Habitação) pousam nas zonas B e A. M06 e M03 são adiados porque a zona C está ocupada; M05 é adiado pela falha de sensor e porque a zona B já foi tomada por M02. M04 é abortado: com 23% de combustível não pode nem descer nem aguardar. A pilha de eventos lista as seis decisões do mais recente ao mais antigo.

5. Modelagem matemática: altitude durante a descida

O fenômeno escolhido é a altitude do módulo em função do tempo durante a fase de descida sem frenagem, entre o fim da aerofrenagem e o acionamento dos retrofoguetes. Nessa fase o módulo cai sob a gravidade de Marte, e a altitude é uma **função quadrática** do tempo:

$$h(t) = h_0 - v_0 t - \frac{g}{2} t^2$$

- h_0: altitude no início da fase (2.000 m no cenário);
- v_0: velocidade vertical inicial, já reduzida pelo paraquedas (60 m/s);
- g: aceleração da gravidade em Marte, 3,71 m/s² (cerca de 38% da terrestre);
- t: tempo decorrido em segundos.

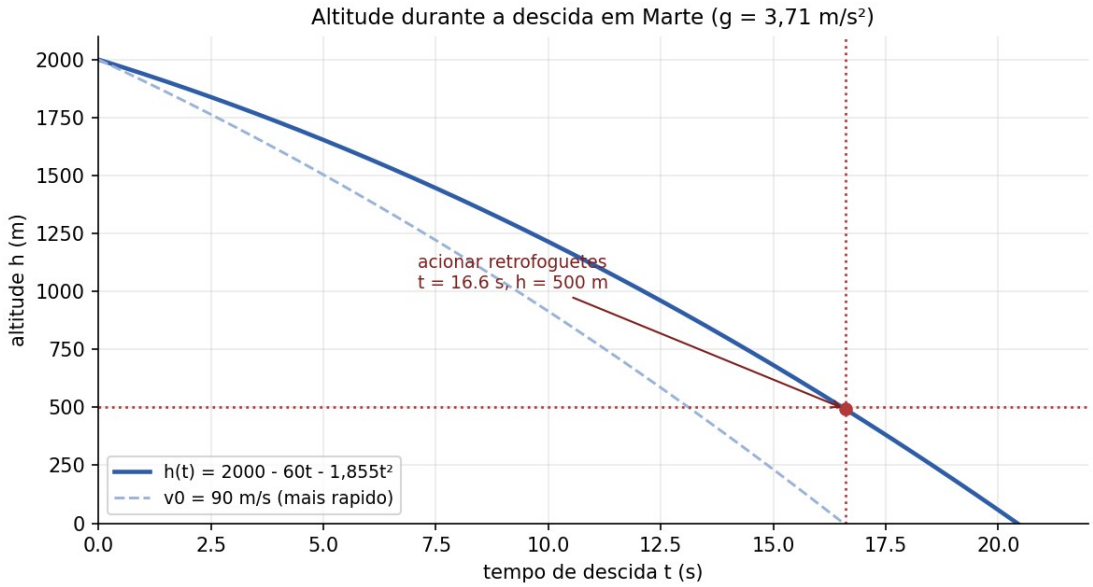


Figura 3. Altitude em função do tempo para dois valores de velocidade inicial.

Análise qualitativa. A parábola tem concavidade para baixo: a altitude cai cada vez mais rápido, porque a velocidade cresce linearmente com o tempo. Quando v_0 aumenta (linha tracejada, 90 m/s), a curva fica mais inclinada e o módulo chega à mesma altitude em menos tempo, reduzindo a janela de reação. Quando g é menor, como em Marte comparado à Terra, a queda é mais lenta, o que dá mais tempo ao sistema de controle, mas também

significa que os retrofoguetes precisam de menos empuxo para frear. Se h_0 aumenta, toda a curva sobe e o tempo até o solo cresce.

Decisão de engenharia. O MGPEB define que os retrofoguetes devem acionar a 500 m de altitude. Resolvendo $h(t) = 500$ com os parâmetros do cenário, obtém-se $t \approx 16{,}5$ s (o protótipo faz essa busca por varredura e encontra 16,6 s). Esse é o instante programado no computador de bordo. A mesma função mostra que, sem frenagem, o módulo tocaria o solo em cerca de 20,5 s, ou seja, há apenas 4 s entre o acionamento e o impacto, o que justifica a regra de abortar pousos com combustível abaixo do mínimo: não há margem para uma segunda tentativa. O modelo também orienta quantos módulos podem descer ao mesmo tempo: como cada descida ocupa uma zona por cerca de 20 s mais o tempo de estabilização, o MGPEB libera no máximo um módulo por zona livre.

6. O MGPEB e a evolução da computação

Os primeiros computadores de propósito geral, como o ENIAC (1945), ocupavam salas inteiras, consumiam dezenas de quilowatts e eram programados por cabos. A substituição das válvulas por transistores nos anos 1950 e, depois, pelos circuitos integrados nos anos 1960 permitiu reduzir tamanho, consumo e taxa de falhas ao mesmo tempo. O marco dessa trajetória para a exploração espacial foi o Apollo Guidance Computer (1966): cerca de 2 KB de memória de trabalho, 36 KB de programa gravado fisicamente em núcleos de ferrite e um escalonador por prioridades que, durante o pouso da Apollo 11, descartou tarefas de baixa prioridade para manter o controle da descida. Foi o primeiro computador embarcado de alta confiabilidade, e é o antepassado direto do MGPEB.

Em Marte, as limitações de hardware continuam severas. Os rovers Curiosity e Perseverance usam o processador RAD750, uma versão endurecida contra radiação de um chip da década de 1990: cerca de 200 MHz e 256 MB de RAM, milhares de vezes menos que um notebook comum. O endurecimento é necessário porque partículas de alta energia invertem bits de memória, o que exige memória com correção de erro, redundância de processadores e código determinístico. A energia também é escassa: cada ciclo de processamento gasta a carga das baterias que o módulo precisa para a frenagem.

Essas restrições explicam as escolhas do MGPEB. Com seis módulos, a ordenação por inserção, $O(n^2)$, executa em poucas dezenas de comparações e não exige memória adicional; um algoritmo mais sofisticado traria complexidade sem ganho prático. A fila e a pilha têm tamanho pequeno e previsível, o que evita alocação dinâmica imprevisível em tempo crítico. As regras de decisão são funções booleanas puras, sem estado oculto, o que as torna verificáveis por tabela verdade e implementáveis diretamente em hardware, como no circuito da Figura 1. Por fim, o modelo de altitude é uma única expressão quadrática, calculável em microssegundos, em vez de uma simulação numérica pesada. Em sistemas embarcados, simplicidade é um requisito de confiabilidade, não uma limitação.

7. Princípios ESG na base Aurora Siger

Escolha da área de pouso. As zonas A, B e C foram definidas por três critérios: distância de regiões com possível presença de água ou de interesse astrobiológico, para respeitar os protocolos de proteção planetária e não contaminar amostras futuras; terreno plano e com baixa densidade de poeira, para reduzir o risco de pouso e o desgaste dos equipamentos; e exposição solar, para a geração de energia. A ordem de pouso do MGPEB (energia e habitação primeiro) reflete essa prioridade: a base precisa gerar energia e abrigar a tripulação antes de expandir.

Energia, resíduos e recursos locais. A energia vem de painéis solares e da carga restante dos módulos, com um orçamento diário que o MGPEB considera ao limitar quantos módulos descem por vez. Resíduos não têm destino fora da base: embalagens e materiais devem ser projetados para reutilização, e a água é reciclada em ciclo fechado. O uso de recursos locais (gelo, regolito para blindagem contra radiação) deve ser inventariado e limitado por cotas, para que a colônia não repita na fronteira marciana o padrão extrativista que degradou ambientes na Terra.

Governança, transparência e participação. Toda decisão do MGPEB fica registrada na pilha de eventos, com o módulo, a decisão e os motivos. Esse registro é a base de três mecanismos: auditoria (qualquer decisão pode ser reconstituída a partir das regras e da telemetria), transparência (a tripulação e o controle na Terra têm acesso ao mesmo registro) e participação (as regras e os limiares, como o combustível mínimo e o vento máximo, são definidos em conjunto e revisados por um conselho que inclui a tripulação, e não apenas pela equipe de engenharia). Em um ambiente de fronteira, onde a comunicação com a Terra leva minutos e a pressão por resultados é alta, regras explícitas e registros imutáveis são a principal proteção contra decisões arbitrárias.

8. Conclusão

O MGPEB mostra que um sistema de pouso pode ser descrito com conteúdos fundamentais: cinco variáveis booleanas e três portas lógicas decidem o pouso; uma fila, três listas e uma pilha organizam os módulos e a auditoria; duas buscas e duas ordenações priorizam quem desce; e uma função quadrática define o instante de acionar os retrofoguetes. No cenário simulado, dois módulos pousam, três aguardam e um é abortado, com todos os motivos registrados. As limitações de hardware de uma missão em Marte não são obstáculo a esse projeto, e sim a razão de ele ser simples.

Anexo A. Estruturas de dados no projeto

Lista (MODULOS, pousados, em_espera, em_alerta). A lista é a estrutura de cadastro: cada módulo é um dicionário com seus atributos, e a lista MODULOS guarda os seis. Buscas e ordenações recebem uma lista e devolvem uma nova lista, sem alterar a original. As três listas auxiliares recebem os módulos conforme a decisão; por exemplo,

após a simulação, pousados contém M02 e M01, e em_alerta contém M04. Como a lista permite acesso por índice, ela é a base da busca binária.

Fila (classe Fila). A fila modela a ordem de atendimento: o primeiro módulo a entrar é o primeiro a ser avaliado (FIFO). A classe tem enfileirar (adiciona no fim), desenfileirar (remove do início), primeiro, vazia e tamanho. Na simulação, a fila final é montada com a lista ordenada por prioridade e combustível (M02, M01, M06, M05, M03, M04) e esvaziada com while not fila.vazia(): m = fila.desenfileirar(). Isso garante que um módulo de prioridade 4 nunca seja avaliado antes de um de prioridade 1, mesmo que tenha chegado à órbita primeiro.

Pilha (classe Pilha). A pilha modela o registro de eventos: o último evento é o primeiro a ser consultado (LIFO). A classe tem empilhar, desempilhar, topo, vazia e tamanho. A cada decisão, eventos.empilhar((m["id"], decisao)) registra o par módulo e decisão. Ao final, eventos.topo() devolve o evento mais recente, ('M04', 'ABORTADO'), e o laço de desempilhar imprime o histórico em ordem inversa. Essa é a ordem natural de uma investigação: começar pela última decisão e voltar no tempo. A mesma pilha permitiria desfazer uma decisão, retirando o evento do topo e devolvendo o módulo à fila.

Estrutura	Papel no MGPEB	Operações usadas	Ordem
Lista	cadastro, resultados de busca e ordenação, listas auxiliares	índice, percorrer, acrescentar	qualquer
Fila	fila de pouso	enfileirar, desenfileirar, primeiro	FIFO
Pilha	registro de eventos para auditoria	empilhar, desempilhar, topo	LIFO

Tabela 4. Resumo das estruturas lineares no projeto.